

Experiment - 6

Create a stored procedure `transfer_employee(emp_id, new_dept_id)` with validation and error handling. Implement triggers for salary validation and audit logging. Test edge cases.

1. Database and Schema Setup (DDL & DML)

-- Step 1: Create and select the database

```
CREATE DATABASE IF NOT EXISTS LabPracticalDB; USE
LabPracticalDB;
```

-- Step 2: Create Department Table

```
CREATE TABLE Department ( dept_id
INT PRIMARY KEY, dept_name
VARCHAR(50) NOT NULL
);
```

-- Step 3: Create Employee Table (with manager_id for hierarchy)

```
CREATE TABLE Employee ( emp_id INT
PRIMARY KEY, emp_name
VARCHAR(100) NOT NULL,
salary DECIMAL(10, 2),
dept_id INT,
manager_id INT,
FOREIGN KEY (dept_id) REFERENCES Department(dept_id),
FOREIGN KEY (manager_id) REFERENCES Employee(emp_id)
);
```

-- Step 4: Insert Departments

```
INSERT INTO Department (dept_id, dept_name) VALUES
(101, 'Engineering'),
(102, 'Data Science'),
(103, 'Human Resources');
```

-- Step 5: Insert Employees (Notice the manager_id assignments establishing a chain of command)

-- Employee 1 is the CEO (Manager is NULL)

```
INSERT INTO Employee (emp_id, emp_name, salary, dept_id, manager_id) VALUES
(1, 'Aarav Sharma', 150000, 101, NULL),
(2, 'Vihaan Singh', 120000, 101, 1),
(3, 'Ananya Gupta', 115000, 102, 1),
(4, 'Advik Desai', 85000, 101, 2),
(5, 'Myra Reddy', 95000, 102, 3),
(6, 'Diya Kumar', 75000, 103, 1),
(7, 'Ishaan Mehta', 65000, 101, 4),
(8, 'Kabir Das', 60000, 102, 5);
USE LabPracticalDB;
```

```
CREATE TABLE Employee_Audit (
SHIVAGANESH
```

```

    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT,    old_dept_id INT,    new_dept_id INT,
    old_salary DECIMAL(10,2),    new_salary
    DECIMAL(10,2),
    action_type VARCHAR(50),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

2. Implement Triggers

Trigger 1: Salary Validation This trigger fires BEFORE an update is made to the Employee table. It intercepts the query and cancels it if the new salary is negative.

```

DELIMITER //

CREATE TRIGGER Before_Employee_Update
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    -- Prevent setting a negative salary
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Validation Error: Salary cannot be a negative value.';
    END IF;
END //
DELIMITER ;

```

Trigger 2: Audit Logging This trigger fires AFTER an update is successfully made. It records the old and new values into the Audit table.

```

DELIMITER //

CREATE TRIGGER After_Employee_Update
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    -- Insert a record into the audit log whenever a department or salary changes
    INSERT INTO Employee_Audit
        (emp_id, old_dept_id, new_dept_id, old_salary, new_salary, action_type)
    VALUES
        (OLD.emp_id, OLD.dept_id, NEW.dept_id, OLD.salary, NEW.salary, 'EMPLOYEE
UPDATE'); END //

```

```

DELIMITER ;

```

3. Create the Stored Procedure

This procedure handles the transfer of an employee with explicit error handling (using a SQLEXCEPTION handler to roll back the transaction) and input validation.

```

DELIMITER //

```

```

CREATE PROCEDURE transfer_employee(IN p_emp_id INT, IN p_new_dept_id INT)
BEGIN
    DECLARE v_emp_exists INT DEFAULT 0;
    DECLARE v_dept_exists INT DEFAULT 0;
    DECLARE v_current_dept INT;
    -- Error Handler: If any SQL error occurs, roll back the transaction
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Transaction Failed: An unexpected database error occurred.' AS Error_Message;
    END;
    START TRANSACTION;
    -- VALIDATION 1: Check if Employee exists
    SELECT COUNT(*) INTO v_emp_exists FROM Employee WHERE emp_id = p_emp_id;
    IF v_emp_exists = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Employee ID does not exist.';
    END IF;

    -- VALIDATION 2: Check if Department exists
    SELECT COUNT(*) INTO v_dept_exists FROM Department WHERE dept_id = p_new_dept_id;
    IF v_dept_exists = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Target Department ID does not exist.';
    END IF;

    -- VALIDATION 3: Check if Employee is already in the target department
    SELECT dept_id INTO v_current_dept FROM Employee WHERE emp_id = p_emp_id;
    IF v_current_dept = p_new_dept_id THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Employee is already assigned to this
department.';
    END IF;

    -- EXECUTE: If all validations pass, perform the update
    UPDATE Employee
    SET dept_id = p_new_dept_id
    WHERE emp_id = p_emp_id;

    COMMIT;
    SELECT CONCAT('Success: Employee ', p_emp_id, ' transferred to Department ', p_new_dept_id) AS
Status_Message;
END //
DELIMITER ;

```

4. Testing Edge Cases (DML)

Run these test cases one by one in MySQL Workbench and take screenshots of the output/errors for your practical file.

Test A: Trigger Validation (Negative Salary) This will fail due to the Before_Employee_Update trigger.

```
UPDATE Employee SET salary = -5000 WHERE emp_id = 1;
```

The screenshot displays the byteXL SQL editor interface. On the left, the SQL code for a procedure named `transfer_employee` is visible. The procedure includes several validation steps: checking if the employee exists, if the target department exists, and if the employee is already in the target department. It then performs an update and commits. An audit log is generated using `CONCAT`. The right pane shows the output of the procedure, including a table of employees and their hierarchy, and an error message at the bottom: "ERROR 1644 (45000) at line 188: Validation Error: Salary cannot be a negative value."

Test B: Procedure Validation (Employee doesn't exist) This will fail because employee 999 does not exist.

CALL transfer_employee(999, 102);

Test C: Procedure Validation (Department doesn't exist) This will fail because department 999 does not exist.

CALL transfer_employee(1, 999);

Test D: Procedure Validation (Already in department) (Assuming Employee 1 is currently in department 101)

CALL transfer_employee(1, 101);

Test E: Successful Transfer & Audit Log Check

Successful Transfer & Audit Log Check This will successfully transfer the employee, and the After_Employee_Update trigger will silently log it in the background.

-- 1. Execute the successful transfer

CALL transfer_employee(1, 102);

-- 2. Verify the base table was updated

SELECT emp_id, emp_name, dept_id FROM Employee WHERE emp_id = 1;

-- 3. Verify the Trigger successfully logged the event in the Audit table

SELECT * FROM Employee_Audit;

VS Code AI Agent

87%

STATUS

Input for the program (Optional)

Output

4	Advik Desai	Vihaan Singh
5	Myra Reddy	Ananya Gupta
6	Diya Kumar	Aarav Sharma
7	Ishaan Mehta	Advik Desai
8	Kabir Das	Myra Reddy

emp_name	salary
Ishaan Mehta	70000.00

emp_id	emp_name	dept_id	hierarchy_level	chain_of_command
1	Aarav Sharma	102	1	Aarav Sharma
2	Vihaan Singh	102	2	Aarav Sharma -> Vihaan Singh
3	Ananya Gupta	102	2	Aarav Sharma -> Ananya Gupta
4	Diya Kumar	102	3	Aarav Sharma -> Diya Kumar
5	Advik Desai	102	3	Aarav Sharma -> Vihaan Singh -> Advik Desai
6	Myra Reddy	102	3	Aarav Sharma -> Ananya Gupta -> Myra Reddy
7	Ishaan Mehta	102	4	Aarav Sharma -> Vihaan Singh -> Advik Desai -> Ishaan Mehta
8	Kabir Das	102	4	Aarav Sharma -> Ananya Gupta -> Myra Reddy -> Kabir Das

Status_Message
Success: Employee 1 transferred to Department 102

emp_id	emp_name	dept_id
1	Aarav Sharma	102

Light

SaveHistorySave